# Ontology Reasoning with GestaltDB

***Release 0.12.1***

**Mylonas Charilaos**

**Oct 22, 2026**

# TUTORIAL


**1 Property graphs vs semantic graphs** **3**
- 1.1 Stage 1 — the property-graph world . . . 3
- 1.2 Stage 2 — the semantic overlay . . . 3
- 1.3 Stage 3 — the reasoning layer . . . 6
- 1.4 Theory and further reading . . . 6

**2 The gestalt-core-v1 fragment and its compiler** **9**
- 2.1 Theory and further reading . . . 9

**3 Datalog IRs: terms, rules, plans, batches** **12**
- 3.1 Theory and further reading . . . 13

**4 Storage grounding: from facts to pinned reads** **14**

**5 Claims, time, and belief** **15**
- 5.1 Theory and further reading . . . 15

**6 Querying and sampling with inferred facts** **17**
- 6.1 Theory and further reading . . . 17

**7 Performance characteristics (advanced)** **18**
- 7.1 A useful cost model . . . 19

**8 Gaps and pain points** **20**

**9 Tutorial toolkit** **22**
- 9.1 Equation conventions . . . 22

**10 Tutorial notebooks** **24**

**11 Outline coverage** **25**
- 11.1 Review corrections (2026-10-22) . . . 25
- 11.2 Graphics and PDF (2026-10-22) . . . 26

An ontology-reasoning tutorial for advanced coders, grounded in the `gestalt-core-v1` fragment on `feature/onto-dev-epic-2`.

This tutorial is (1) a basic ontology-reasoning walkthrough with gestaltdb, (2) an introduction to the intermediate representations and foundational capabilities underneath — and how gestaltdb supports them efficiently — and (3) an explicit map of gaps and pain points, linked to open issues.

Start with chapters 1–2 and notebook 1 for a first inference. Chapters 3–6 explain the execution and storage boundaries; chapters 7–8 are the performance and limitations reference. The *toolkit* regenerates the illustrations, and the *notebooks* provide runnable labs.

### Setup and version boundary

Examples target this source checkout, based on ontology integration commit `0dba88f`, not the published package on PyPI. On 2026-10-22, remote `main` was `25daaf9` and remote `feature/onto-dev-epic-2` was `9400930`; the latter contains additional work not imported into this tutorial branch. An open roadmap issue is not evidence that its code is absent: chapter 8 separates available capabilities from outstanding acceptance work.

From the repository root, use a supported Python (3.10–3.13) and `uv`:

```
uv sync --extra lmdb
uv run --with jupyterlab --with ipykernel jupyter lab docs/onto-
↪tutorial/notebooks
./scripts/build_docs.sh
```

The standalone reasoning examples need no embedded backend. The property-graph labs use the explicit `lmdb` extra, temporary databases, and deterministic assertions. Choose the notebook kernel belonging to this environment; installing Jupyter elsewhere does not install GestaltDB into that kernel.

Build output starts at `docs/_build/html/onto-tutorial/index.html`. Building Sphinx renders prose and figures; it does not execute notebook cells. Run the tutorial's executable checks with:

```
uv run pytest tests/test_ontology_tutorial.py -q
```

### PDF export

Build HTML plus a standalone tutorial PDF (not the whole API reference):

```
./scripts/build_docs.sh --pdf
```

The PDF is written to `docs/_build/html/onto-tutorial/onto-tutorial.pdf`. With `--output DIR`, find it at `DIR/onto-tutorial/onto-tutorial.pdf`; LaTeX intermediates and diagnostic logs are alongside it in `latex/`. Math is typeset by XeLaTeX, diagrams remain vector artwork, and the document includes a table of contents and clickable external links. Notebook downloads remain separate executable labs; notebook outputs are not executed into the PDF.

PDF export needs XeLaTeX and Sphinx's standard LaTeX packages. For example, on Debian/Ubuntu install `texlive-xetex`, `texlive-latex-extra`, `texlive-fonts-recommended`, and `fonts-dejavu-core`. The build command provisions its Python dependencies via `uv`. HTML-only builds do not require TeX, CairoSVG, or Graphviz. PDF conversion uses CairoSVG and its system Cairo library; regenerating graph layouts separately requires Graphviz's `dot`.

CHAPTER

ONE

# PROPERTY GRAPHS VS SEMANTIC GRAPHS

**Intuition first.** Picture a whiteboard: sticky notes for things (`alice`, `acme`), arrows between them labeled `worksFor`. That is a *property graph* — gestaltdb's home turf (`src/gestaltdb/graphdb.py`): `Node` objects carry native labels, `Edge` objects carry their relationship in the `properties["type"]` convention. Now lay *tracing paper* over the whiteboard and copy every arrow as a uniform triple `(subject, predicate, object)`; write class membership as just another arrow, `(alice, rdf:type, Researcher)`. That tracing is the *semantic* view. This tutorial's recurring diagram is modeled as a real GestaltDB `Node/Edge` graph in `toolkit/landscape.py`. Graphviz's layered layout places large rounded nodes and routes curved, labeled arrows while reducing crossings. Colors and enclosing groups distinguish conceptual layers; small badges describe each node's role. It grows in three stages.

## 1.1 Stage 1 — the property-graph world

`Property Graph DB` *stores* labeled nodes and typed edges. **Cypher** expresses graph patterns (GestaltDB's query language; see ../cypher). **GraphQL** describes a typed API response: application resolvers may read a graph, a relational database, or remote services. It is not a native property- graph query language. **SQL** queries relations; **SQL/PGQ** adds property-graph pattern matching to SQL implementations that support it. GestaltDB supplies neither SQL/PGQ nor a GraphQL server. The map compares technologies, not a list of supported endpoints.

Intuition: a query language is a way to ask about the whiteboard; an API is a way to package answers. Naming an arrow `worksFor` does not by itself add domain, range, or subclass inference.

## 1.2 Stage 2 — the semantic overlay

Selected property nodes/edges *project to* binary facts with an explicit mapping. **RDF** represents graphs as sets of triples: subjects are IRIs or blank nodes, predicates are IRIs, and objects may also be literals. A bare `Fact("alice", "worksFor", "acme")` is a logical triple over local symbols, not automatically a standards-compliant RDF serialization. Export would need an IRI policy. Parallel edges collapse under set semantics unless their identities are modeled separately.

**OWL** provides an ontology language, with an RDF serialization, for saying what relationships mean (subclasses, domains, inverses, transitivity). A gestaltdb **Ontology** *selects from* OWL a small decidable fragment (`PROFILE =`

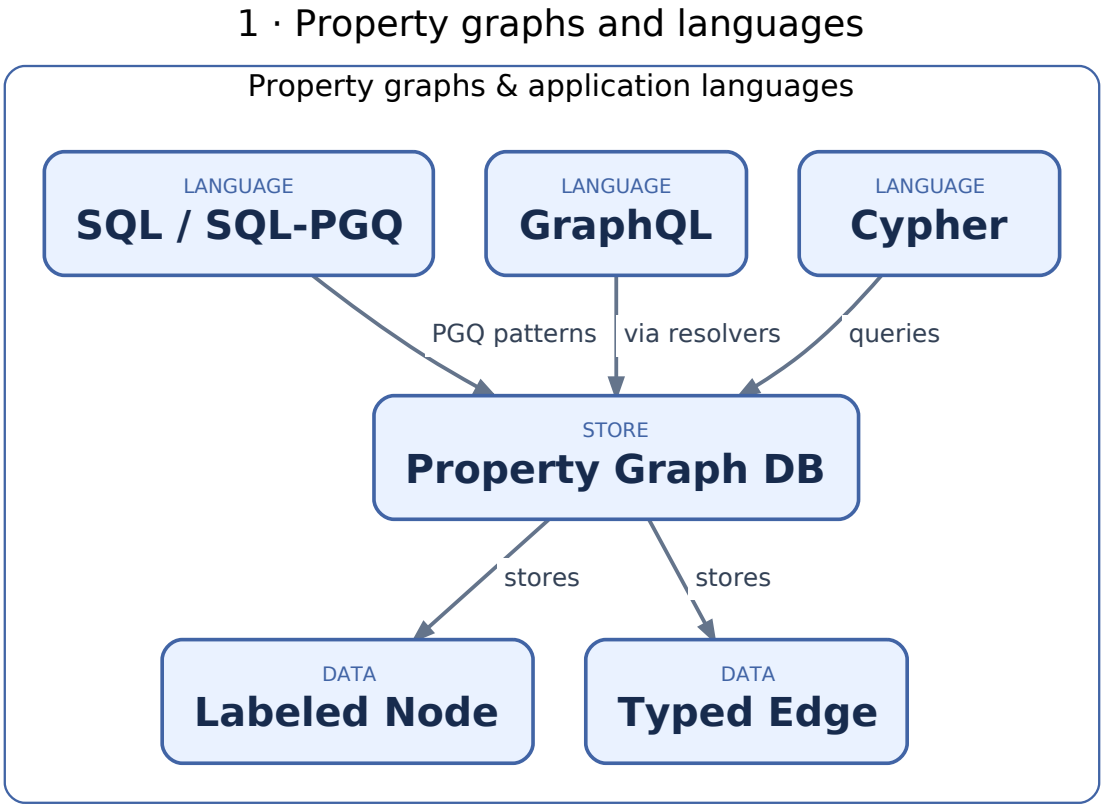



Fig. 1: Stage 1: the store and its query languages.

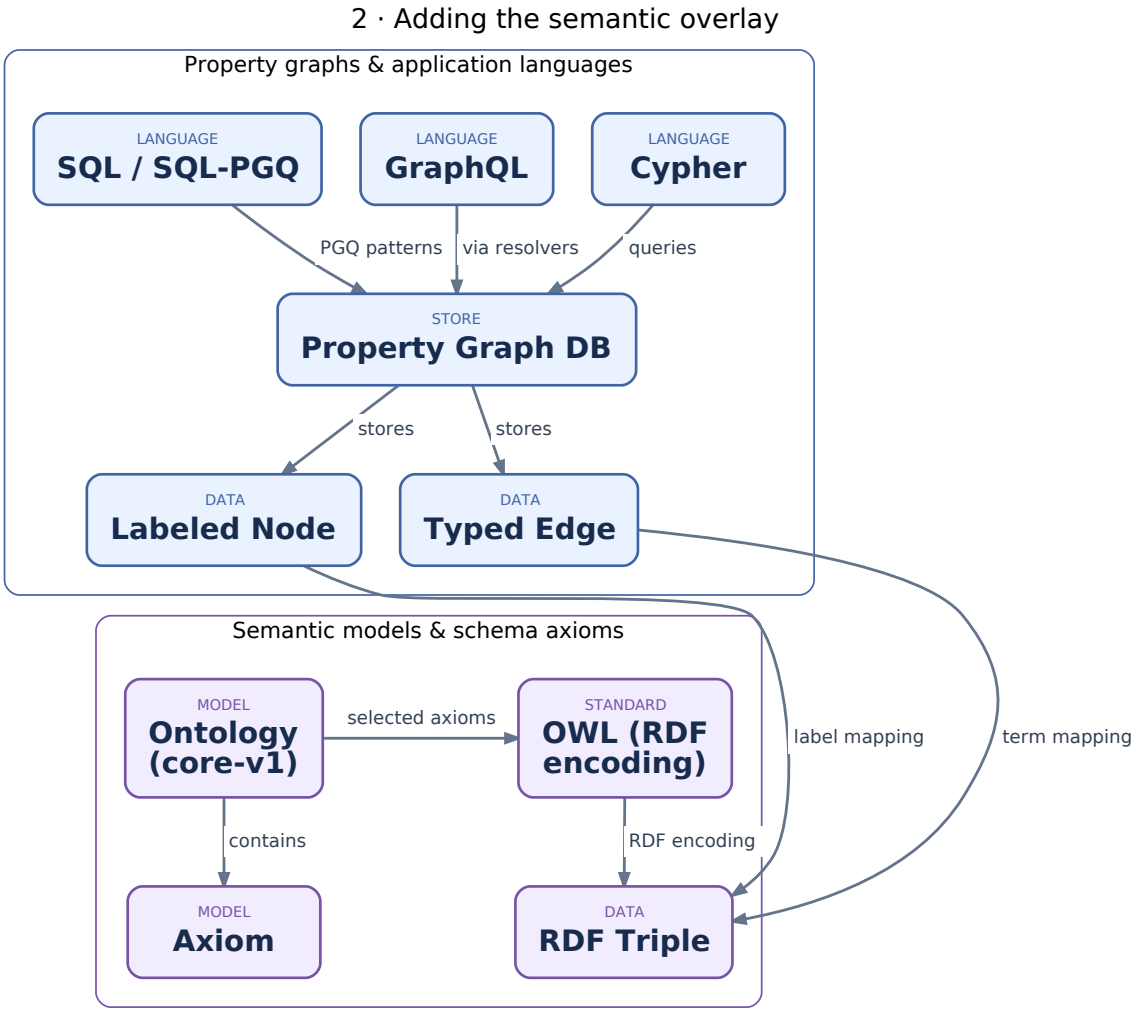



Fig. 2: Stage 2: projecting the whiteboard onto tracing paper.

"gestalt-core-v1") and *contains* **Axioms** — see *The gestalt-core-v1 fragment and its compiler*. Intuition: the tracing paper doesn't replace the whiteboard; it is an explicit, lossy-but-uniform copy. The projection is caller-owned. `reasoning/adapters/graph.py` maps current graph records to facts. In contrast, `ontology/adapters/rdf.py` imports selected *schema axioms* from RDF documents; it is not a general instance-data (ABox) loader. Strict import rejects unsupported constructs. Non-strict import explicitly marks the report incomplete and requires separate consent before compilation.

## 1.3 Stage 3 — the reasoning layer

Axioms *compile to* **Rules** over **Facts**; **Datalog** *defines* that rule shape (safe, function-free Horn rules — see *Datalog IRs: terms, rules, plans, batches*). **Prolog** is a related logic-programming language, typically evaluated by goal-directed search with clause order, compound terms, and optional control features. It is not simply Datalog with more keywords. `DatalogEngine` evaluates facts or evidence supplied by callers; explicit adapters can obtain those from current records or temporal claims. The engine does not read a database or construct a `SamplerSnapshot` itself. Passing inferred endpoints as sampler seeds does not make later hops traverse inferred edges.

**Cypher** and Datalog both bind variables by matching relations, but their contracts differ: Cypher returns query rows (and can perform writes), whereas positive Datalog computes a set-valued closure. Intuition: the engine reads the tracing paper plus the rulebook and pencils in every arrow the rules demand (`alice` is a `Person`), while the whiteboard itself stays untouched.

Parallels: LPG systems (Neo4j/Cypher) keep labels and types as queryable data; RDF/OWL stores (RDFLib, OWL-RL, Jena) treat triples plus entailment regimes as the data model. GestaltDB is an LPG store with an opt-in, offline semantic overlay — nothing is silently rewritten.

## 1.4 Theory and further reading

- Graph database and Graph query language survey the LPG/RDF split and the language families (Cypher, Gremlin, SPARQL) that grew around each side.
- Cypher is pattern matching over stored structure; SPARQL is the W3C counterpart over triple stores (SPARQL 1.1 overview). GestaltDB deliberately exposes Cypher but no SPARQL endpoint — inferred triples are consumed through explicit adapters, not a second query language.
- Resource Description Framework (RDF 1.1 Concepts) defines the triple model gestaltdb projects onto; the Semantic Web article gives the original vision (machine-readable web of data) that OWL and the ontology layer come from.
- GraphQL specifies client-shaped API selection; its resolvers choose how and where to fetch the data.

```
from gestaltdb.graphdb import Edge, GraphDB, Node
from tempfile import TemporaryDirectory
```

(continues on next page)

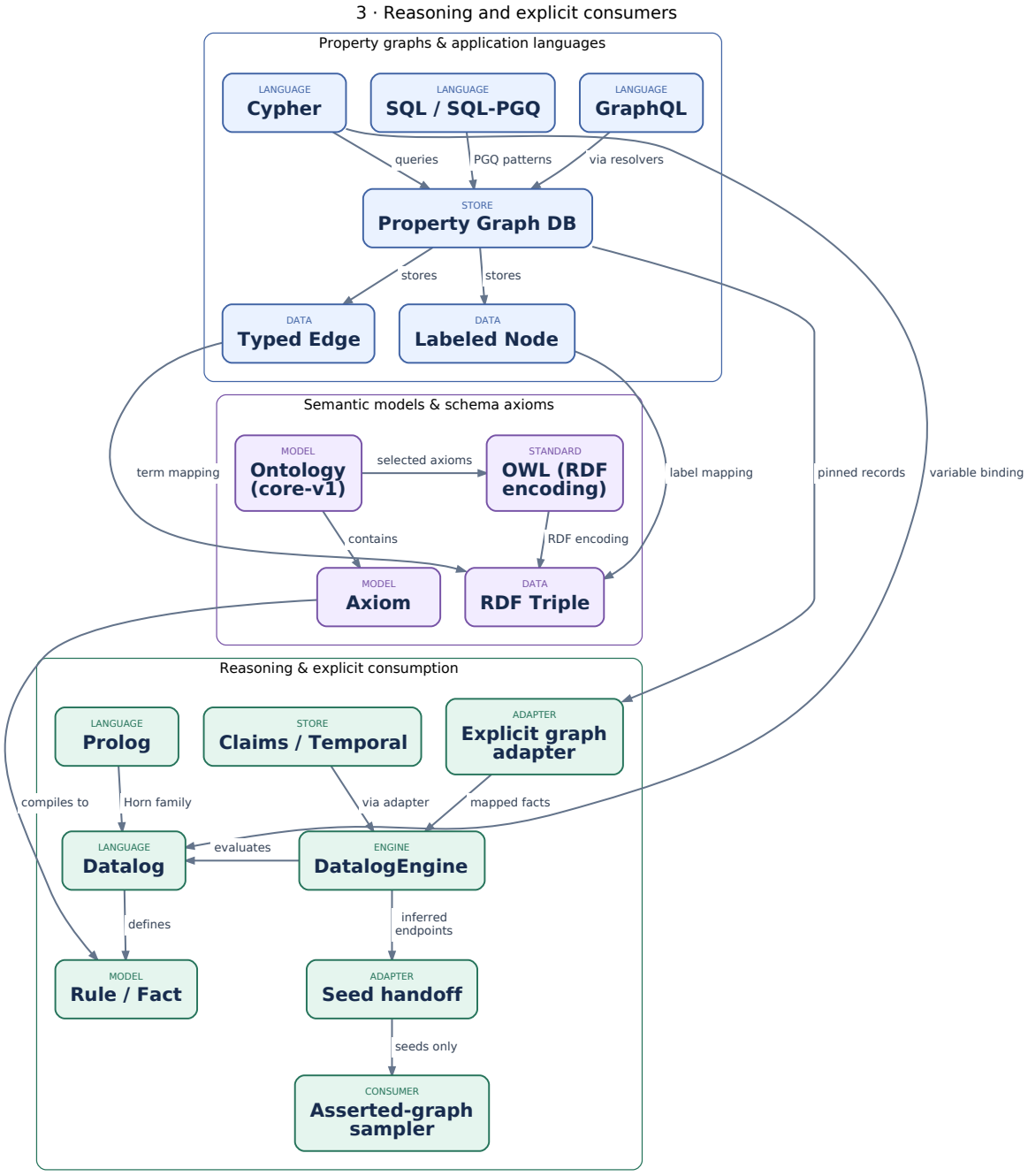



Fig. 3: Stage 3: an assistant that fills in the missing arrows.

(continued from previous page)

```
with TemporaryDirectory() as directory:
    graph = GraphDB.create(directory, backend="lmdb", serializer=
↪"json")
    try:
        graph.put_nodes([
            Node(node_id="alice", labels=["Researcher"],
                 properties={"name": "Alice"}),
            Node(node_id="acme", labels=["Organization"]),
        ])
        graph.put_edge(Edge(edge_id="e1", source="alice", target=
↪"acme",
                            properties={"type": "worksFor"}))
        rows = graph.query(
            'MATCH (a:Researcher) RETURN a.name AS name').records
        assert rows == [{"name": "Alice"}]
        assert not graph.query('MATCH (a:Person) RETURN a').records
    finally:
        graph.close()
```

The second assertion is the important one: without an explicit reasoning step, the stored label `Researcher` does not imply a stored label `Person`.

CHAPTER

TWO

# THE GESTALT-CORE-V1 FRAGMENT AND ITS COMPILER

`gestaltdb.ontology.model` defines immutable axioms: `SubClassOf`, `SubPropertyOf`, `EquivalentClasses/Properties`, `Domain`, `Range`, `InverseOf`, `Symmetric`, `Transitive`, plus `PropertyChain`, `IntersectionSubClassOf` / `UnionSubClassOf` (left-side only) and scoped `Reflexive`. `PROFILE = "gestalt-core-v1"`. Each axiom carries a content `axiom_id` (`sha256` over `["gestaltdb-axiom-v1", signature]`).

`compile_ontology(Ontology(id, axioms))` (`ontology/compiler.py`) lowers each axiom to safe positive Horn rules with `rule_id = ontology:{axiom_id}:{direction}` and a `RuleOrigin` row per rule. The `CompiledOntology` checks that origins cover exactly the source axioms.

$$\frac{(x, \texttt{rdf:type}, C)}{(x, \texttt{rdf:type}, D)}\ (C \sqsubseteq D) \qquad \frac{(x, p, y)}{(x, q, y)}\ (p \sqsubseteq q) \qquad \frac{(x, p, y)\ \ (y, p, z)}{(x, p, z)}\ (\text{Transitive}(p))$$

Rejected (fail closed): disjunctive heads, unscoped reflexivity, chains of length != 2, `rdf:type` as a property, right-side unions. This is *not* full RDFS/OWL — see ontology and gated profile work (#184, #185).

## 2.1 Theory and further reading

- Description logic studies the decidable fragments of first-order logic behind OWL (`ALC`, `SHOIN`, `SROIQ`). Gestalt-core-v1 admits only its selected finite Horn rewrites, which is why this fragment needs no tableau reasoner.
- Web Ontology Language (OWL 2 overview) standardizes the full vocabulary (including existentials, cardinality, and keys). The named-entity compiler used here does not support that whole vocabulary. Separate experimental modules do not expand its contract: consult ../reasoning-profiles and ../reasoning-el rather than assuming full OWL support.
- Horn clause — at most one positive literal — is the logical shape every compiled rule must have. Disjunctive conclusions are not single-head Horn rules. Unscoped reflexivity is a different restriction: it needs an explicit finite domain of individuals here, not a different logical clause shape.

**Intuition.** Read the picture below in two groups: blue is *data* (what someone asserted), purple is *schema* (what the arrows mean). `domain` says "anything a `worksFor` arrow leaves is a `Researcher`"; `subClassOf` says "every `Researcher` is also a `Person`". The `inferred type` arrows are computed consequences, not stored labels. Both `Researcher` and `Person` memberships follow from the one asserted `worksFor` fact:

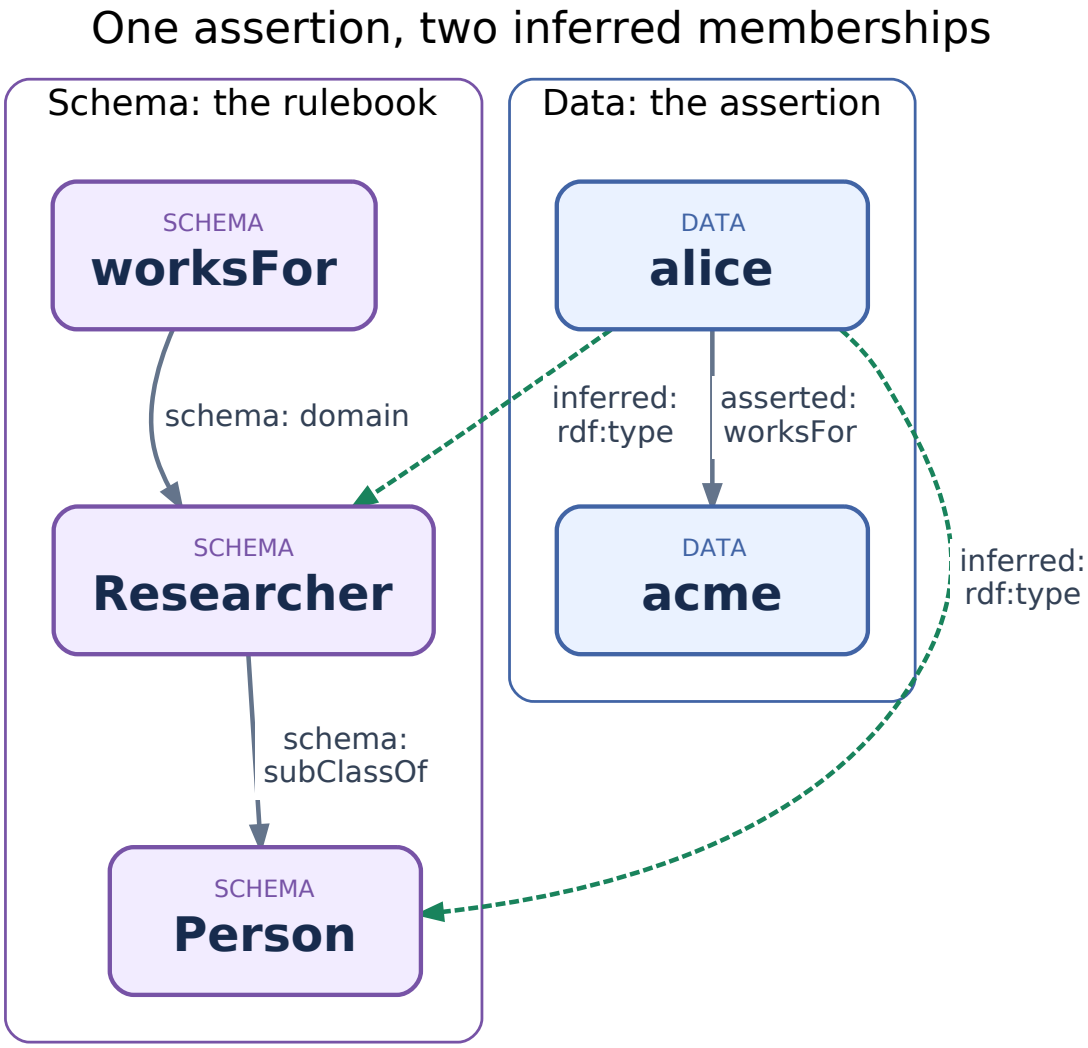



Fig. 1: Schema and data are visually separate; memberships are computed in this example, not written back. Explicit publication is a separate API.

```
from gestaltdb.ontology import Ontology, SubClassOf, Domain,␣
↪compile_ontology
from gestaltdb.reasoning import DatalogEngine, Fact, ReasoningLimits
from gestaltdb.ontology import TYPE_PREDICATE

plan = compile_ontology(Ontology("people", [
    Domain("worksFor", "Researcher"),
    SubClassOf("Researcher", "Person"),
]))
asserted = Fact("alice", "worksFor", "acme")
result = DatalogEngine().evaluate(
    plan.program, [asserted],
    limits=ReasoningLimits(), collect_proofs=True)
researcher = Fact("alice", TYPE_PREDICATE, "Researcher")
person = Fact("alice", TYPE_PREDICATE, "Person")
assert result.facts == tuple(sorted((asserted, researcher, person)))
assert not result.explain(person).truncated
assert {origin.axiom_id for origin in plan.origins} == {
    axiom.axiom_id for axiom in plan.ontology.axioms
}
```

Read this as a two-step explanation: the domain rule derives `Researcher`; the subclass rule uses it to derive `Person`. `result` retains the asserted fact as well as both conclusions. Its explanation is one bounded witness, not a complete inventory of alternative supports for deletion maintenance.

**Note**

Domain is an implication, not an input-validation constraint. An untyped subject of `worksFor` gains `Researcher` membership; it is not rejected for lacking that label. Short names in this example are exact local symbols: `Person` and `person` are different constants, and only `TYPE_PREDICATE` supplies the full reserved `rdf:type` IRI.

CHAPTER

THREE

# DATALOG IRS: TERMS, RULES, PLANS, BATCHES

Core values (`reasoning/model.py`): `Variable("x")` (a plain `"?x"` string is a constant), `Fact(subject, predicate, object)` (ground binary entity triple), `Atom` (predicate constant, subject/object terms), `Rule(rule_id, body, head)` (safe: every head variable occurs in the body), `Program + ReasoningLimits`. The standalone `DatalogEngine` (`reasoning/engine.py`) does semi-naive fixpoint; witnesses come from `reasoning/proofs.py`.

Execution IRs (roadmap #156, RE-01..RE-06):

- `reasoning/relational.py` — n-ary-ready relational vocabulary and `explain_plan` (structural only, not executable beyond binary entities).
- `reasoning/batches.py` — compact `uint64` symbol dictionaries + immutable column batches; optional Arrow exchange in `reasoning/adapters/arrow.py` (imports copy and canonicalize; little-endian exports can share immutable column payloads, but wrappers allocate and consumers may copy).
- `reasoning/plans.py` — positive-predicate SCC schedules, delta sites, optional witness/telemetry (off by default).
- `reasoning/adapters/polars.py` — experimental closure-only Polars prototype; `reasoning/executors.py` keeps indexed Python the default and routes proofs to indexed engines.

$$I_0 = F, \qquad I_{n+1} = I_n \cup \{\, h\sigma \mid (B \Rightarrow h) \in P,\ B\sigma \subseteq I_n \,\}, \qquad C_P(F) = \bigcup_{n \geq 0} I_n$$

Here `F` is the input fact set, `P` the rules, and `σ` a substitution of variables by constants. Starting from the empty set without injecting `F` would lose the assertions and derive nothing for our example. With a finite constant vocabulary, fixed predicate arities, safe rules, and no value-generating functions, only finitely many ground facts can be added. Set semantics therefore gives termination for this positive fragment. Demand evaluation (`reasoning/demand.py`, magic-set specialization) answers bound goals without full materialization; typed literals live in `reasoning/terms.py` / `reasoning/typed.py`.

**Intuition.** Semi-naive evaluation considers rule-body joins in which at least one body occurrence comes from the latest delta; other occurrences can match the accumulated relation. It is not enough to join only new facts with each other. Subtracting the existing closure produces the next delta. For chapter 2: round 0 contains `worksFor(alice, acme)`, round 1 adds `type(alice, Researcher)`, round 2 adds `type(alice, Person)`, and the next round adds nothing. SCC plans (`reasoning/plans.py`) group mutually recursive rules so each group reaches its

own fixpoint before downstream rules fire; demand evaluation works backwards from the question ("who is a `Person`?") and only explores rules that could produce an answer.

## 3.1 Theory and further reading

- Datalog fixes the language: function-free Horn rules with set semantics. Prolog relaxes it toward a full programming language. Its usual left-to-right, depth-first evaluation can diverge even for a logically valid recursive specification; cuts and negation as failure also need operational care. This tutorial instead uses bottom-up finite set semantics, not a Prolog execution model.
- Least fixed point and the Knaster–Tarski theorem justify the closure equation above: the inflationary consequence operator is monotone over the finite powerset of ground facts. Iterating from `F` reaches the least model containing the assertions. Monotonicity guarantees a least fixed point; finiteness additionally makes this iteration terminate.
- Magic sets is the rewriting behind demand evaluation: bound arguments propagate sideways so the fixpoint only explores facts relevant to the query, with full-closure answer parity with full closure for the supported fragment. This does not promise lower work for every query or selective reads from the KV backend.
- Stratified negation explains the scoping discipline of `closed` declarations — negation applies per completed stratum, never inside its own recursive loop.

CHAPTER

FOUR

# STORAGE GROUNDING: FROM FACTS TO PINNED READS

Core and compiler never touch storage. Grounding is explicit:

- `reasoning/extraction.py` — bounded `ExtractionRequest` / `FactBatch` contracts with exact version evidence; `reasoning/adapters/batched.py` borrows caller views; LevelDB snapshot requests fail closed.
- `reasoning/adapters/native.py` — audited PyRex Arrow scans, version-gated (`pyrex-rocksdb>=0.5.0`). Raw scans return KV batches; the explicit `iter_decoded_evidence` path supplies decoded evidence with exact version IDs and visibility checks. Issue #194 still requests larger-scale measurements, not the first implementation of decoding.
- `reasoning/adapters/graph.py` — explicit current-graph projection and per-relation fanout (../ontology-graph).

$$\text{extract}(G@h, \varphi) \to \langle \text{Facts}, h \rangle \qquad h = \text{pinned horizon}$$

Point-only publication (`reasoning/adapters/publication.py`, ../ontology-publication) stages generations with compare-and-publish; nontransactional orphan recovery is caller-managed, never silently repaired. Maintenance (`reasoning/maintenance.py`, `catalog.py`) adds staged generations, forward/reverse dependency indexes, insertion propagation and DRed deletion — live-reasoning slice RE-E3 (#159). These modules supply separate contracts, not one automatic storage-to-maintenance service. See ../reasoning-generations, ../reasoning-maintenance, and the status distinctions in *Gaps and pain points*.

CHAPTER

# FIVE

# CLAIMS, TIME, AND BELIEF

Mutable property records (`graphdb.py`), temporal versions (`versioning.py` / `temporal.py`) and epistemic claims (`epistemic.py`) are three separate layers.

- Bitemporal versions: `put_node_version` / `commit_versions` append immutable history; `get_node_as_of` / `iter_edges_as_of` read it. Intervals are half-open `[start, end)` microsecond UTC.
- Claims: `assert_claim` / `correct_claim` / `retract_claim` with deterministic statement IDs; `claim_status` gives open-world `supported` / `refuted` / `both` / `unknown`.
- Modal: `assert_world_accessibility` + `entails` evaluates bounded nested formulas under one read view with depth/state bounds.

$$V_s(h) = \bigcup_{d \in D_s(h)} \bigcap_{p \in \text{premises}(d)} V_p(h), \qquad \text{holdsThroughout}(s, W, h) \iff W \subseteq V_s(h)$$

At a pinned system horizon `h`, `D_s(h)` denotes the admissible positive supports for statement `s`. For this positive-rule illustration, a derivation is valid where *all* premises overlap; alternative derivations contribute a *union*. An assertion supplies its selected valid interval directly. This is not the definition of every window-query policy or of negation/equality. For example, supports covering `[1,2)` and `[2,3)` together cover `[1,3)`, even though neither support alone does. `[1,2)` and `[3,4)` leave a real gap; replacing them by their min/max envelope would invent truth at time 2.5. Truth coverage may coalesce while evidence history keeps both supports.

Temporal closure evaluates change segments, not samples at one instant (`reasoning/temporal_closure.py`, `intervals.py`; slice RE-E4, #160). Confidence-calculus policy contracts are separately gated; they do not give the default evaluator probabilistic semantics (#186, RE-25). Domain/range *entail* memberships; validation (`reasoning/validation.py`) *reports* conflicts without explosion.

## 5.1 Theory and further reading

- Temporal database distinguishes valid time (when the world was so) from transaction/system time (when the database recorded it). GestaltDB keeps exactly this split: valid intervals on versions and a selected system horizon on reads. `verify_closure` checks supported range results against independently recomputed change segments; it is not a general database-recovery verifier.

- Allen's interval algebra catalogues the relations between time intervals (before, meets, overlaps, ...). The tutorial uses concrete half-open `[start, end)` intervals: touching intervals (`[1,2)`, `[2,3)`) share no instant. Allen's qualitative relations are useful background, not a claim that this module implements that algebra.
- Open-world assumption vs closed-world assumption: claims default to open-world (absent evidence means `unknown`, not false); only explicitly `closed` scopes license negation-as-absence (ch03).
- Kripke semantics (possible worlds linked by accessibility relations) is the model behind the modal/belief layer: each agent's frame is a set of worlds, and `entails` checks the formula in the worlds reachable under the depth/state bounds.

CHAPTER

SIX

# QUERYING AND SAMPLING WITH INFERRED FACTS

Evaluation alone does not rewrite the graph. Consumption or publication is explicit:

- `reasoning/adapters/graph.py` — `GraphMapping` + `expand_edge_types` projects inferred predicates to typed traversals with per-relation fanout accounting; `sample_expanded_neighbors` samples without changing arrays.
- Cypher (`graph.query`): no automatic inferred-pattern rewrites. Opt-in call `pg.sample_typed_paths` / `kg.entails` stay allowlisted procedures.
- `SamplerSnapshot` / `SamplerEngine` (`sampling/`): compact integer node/edge/relation IDs, RAM/memmap modes, temporal v2 with point/window filters and causal policies; `node_ids_global` maps local batch rows back to snapshot IDs.

$$N_K(s) = \text{distinct}\left( \bigcup_{r \in R_{exp}} \text{sample}(N_r(s), k_r) \right), \qquad \sum_r k_r = K, \quad |N_K(s)| \le K$$

For `sample_expanded_neighbors`, `N_r(s)` is the distinct logical endpoint set for an expanded member relation, not a bucket of physical edge IDs. Quotas split `K` evenly in sorted relation order, with earlier members taking the remainder. Overlapping endpoints are attributed to the first member; unused quota is not redistributed. Thus fanout 10 is a cap, not a guarantee of ten distinct endpoints or uniform sampling over the union. Multiple proof paths do not increase an endpoint's weight. Inferred seed handoff leaves the next asserted-graph sampling hop unchanged.

Seed filtering alone never constrains every hop — define the eligible graph with snapshot `node_filter` / `edge_filter`. See ../typed-sampling and ../ontology-graph. End-to-end pipeline glue is open work (#255, #256).

## 6.1 Theory and further reading

- SPARQL uses basic graph pattern matching by default. The separate SPARQL 1.1 Entailment Regimes recommendation defines extensions for selected entailment relations. A SPARQL endpoint does not automatically implement OWL inference. GestaltDB exposes Cypher plus explicit reasoning adapters, not a SPARQL endpoint.
- Graph query language compares the traversal idioms (pattern matching, path queries, fanout control) that both Cypher clauses and `expand_edge_types` sampling implement in their own layer.

CHAPTER

SEVEN

# PERFORMANCE CHARACTERISTICS (ADVANCED)

Measure whole pipelines, not kernels. The roadmap (#156) freezes budgets per slice before selecting executors; dense and sparse workloads are benchmarked separately (quadratic closure is not constant-memory).

- Extraction: bounded batches, exact evidence IDs, capability probing; profile extraction separately from fixpoint (RE-05 evidence `benchmarks/results/reasoning-re05-extraction.json`).
- Joins: indexed Python selects binding-index buckets; it is not an unindexed full Cartesian scan. The RE-03 closure-only comparison at source `14ade5a` used Python 3.13 and three fresh workers per cell. Dense N=128 median pipeline times were 46.44ms Polars versus 534.73ms indexed Python; sparse recursive N=1024 also favored Polars (40.40ms versus 90.37ms), whereas hierarchy, duplicate-evidence, and small N=16 cases favored Python. Peak worker RSS was about 127 MiB versus 54 MiB. These are historical workload observations, not current-machine benchmarks or universal speedups. Pipeline timings include compilation, conversion, fixpoint, and decoding, but exclude package imports and fixture construction. See ../reasoning-polars for the tables and reproduction commands; proof and maintenance modes are not equivalent. Arrow imports copy and canonicalize; little-endian exports can share column payloads. Neither means that KV extraction or all downstream operations are zero-copy (../reasoning-batches).
- Plans: SCC schedules + delta sites are reusable architecture, not a proven throughput gain on small programs (RE-04); telemetry off by default.
- Maintenance: DRed vs rebuild crossover measured with P50/P95, RSS, supports and write amplification (RE-18); redundant cycles are no-ops.
- Temporal: interval fragmentation, per-segment evidence, reuse ratios (RE-22); event-sweep scheduling is open (#254).

Always report: cold start, compile, extraction/auth, conversion, join/fixpoint, proofs/supports, publication, decode, total pipeline, peak RSS, thread count, wheel footprint. Cancellation is cooperative (`ReasoningCancelledError`, no partial results); hard deadlines need worker isolation.

## 7.1 A useful cost model

$$T_{\text{total}} = T_{\text{extract}} + T_{\text{compile}} + T_{\text{convert}} + T_{\text{fixpoint}} + T_{\text{evidence}} + T_{\text{publish}}$$

These are conceptual stages, not an assertion that every executor exposes separate timers for each. Speeding up joins cannot remove extraction or proof cost. Materializing reachability on an `n`-node directed chain produces `n(n-1)/2` distinct pairs: quadratic output is inherent even if joins become faster. Compact binary rows use 16 payload bytes per pair before dictionaries, Python staging objects, support indexes, and allocator overhead. Measure both final payload and peak process RSS; they answer different questions.

CHAPTER

EIGHT

# GAPS AND PAIN POINTS

Status reviewed on 2026-10-22 against this checkout (base `0dba88f`), remote `main` (`25daaf9`), remote `feature/onto-dev-epic-2` (`9400930`), and open issues/PRs. No integration-branch merge was performed for this review. Treat the following as a dated capability map, not a release promise.

- Profiles: disjunctive heads, existentials/EL (bounded prototype #185), full OWL RL/DL — construct matrix + approval gates in #184; no badge claims.
- Negation (#171, #125): scoped stratified evaluation exists in the generic engine. The positive binary maintenance API rejects negation/closed scopes, aggregates, changed programs, and equality splits; explicitly rematerialize instead. RE-16 is not a promise of incremental negation support.
- Equality (#172): snapshot-local canonical representatives; literals never merge; no unique-name assumption; temporal representatives rebuild per segment.
- Aggregates: nonrecursive/stratified only; recursive/lattice deferred.
- Extraction (#194): native decoded evidence and locator helpers are present; the remaining acceptance work is larger-scale measurement (100K+ rows and cache/value variation). Small fixtures do not establish a throughput win.
- Integration follow-ups: demand pushdown (#249), resumable commit-delta sessions (#251), assertion multiplicities (#250), generation identity (#248), inferred-generation consumption (#256), and executor pipeline glue (#255) remain active. Draft PRs are not available APIs in this checkout.
- Support-discovery head binding (#252) and immutable dependency lookup caches (#253) have merged implementations in this checkout (PRs #266 and #267), even though their issues remain open. Inspect code/tests and acceptance work rather than equating an open checkbox with an absent capability.
- Scale: profiling and worker/spill policy contracts exist; they are not a deployed out-of-core runtime or a bespoke native executor (RE-26, #187). Bounded one-step minimal/top-k witnesses and authenticated closure checkpoints also exist (RE-27, #188); globally minimal recursive proof trees and durable incremental-session resume are different, stronger requirements. See ../reasoning-proofs and ../reasoning-scale.
- Tracker: techdebt umbrella #200; roadmap phases RE-E1..E5 (#157–#161); neuro-symbolic (#128), weights (#124), explanations (#126) coordinate but stay separate.

Rule of thumb: check each API's declared fragment and supported combinations. The construct matrix in `reasoning/profiles.py` is an ontology approval inventory, not a universal execution-capability dispatcher. A construct's presence there does not make it executable in every engine, adapter, or temporal/maintenance mode. Never silently weaken a requested guarantee.

CHAPTER

NINE

# TUTORIAL TOOLKIT

`toolkit/` contains the tutorial's illustration helpers. The landscape uses GestaltDB graph objects plus the build-time Graphviz `dot` executable, not a browser or an embedded database. Other helpers retain the packaged GestaltDB visualization API. No production-library rendering behavior is changed.

- `figures.py` — a minimal vertical class-graph SVG helper and `save_svg`.
- `viz_helpers.py` — thin wrappers over `gestaltdb.viz.api` with the tutorial's 2000/5000 caps and `edge_labels="all"` default.
- `equations.md` — MyST math conventions ($$ blocks).
- `landscape.py` — the ch01 concept graph as real `Node`/`Edge` objects in 3 progressive stages; `uv run python docs/onto-tutorial/toolkit/landscape.py` regenerates `figures/landscape-stage{1,2,3}.svg` and the worked subsumption figure using Graphviz's hierarchical ranking, crossing reduction, cluster placement, and spline routing. Rounded filled nodes have large bold names, category badges, and padded interiors; relationship labels wrap independently. Dashed green edges mark computed memberships in the worked example. Those inferred membership edges come from `DatalogEngine` rather than a hand-drawn assertion of what it should infer.

Run from the repository root:

```
# Install Graphviz first (e.g. apt install graphviz on␣
↪Debian/Ubuntu).
uv run python docs/onto-tutorial/toolkit/landscape.py
```

No node coordinates are hand-maintained. Committed SVGs keep HTML/PDF builds independent of Graphviz; only regeneration needs `dot` and the DejaVu fonts. Graphviz versions can change placement without changing the graph. Tests check graph content, large text, spline paths, and repeatability on the installed version rather than requiring one version's exact coordinates everywhere. SVG text remains selectable. These static figures do not capture an interactive notebook's browser state. See *Gestalt Graph Onto-Tutorial* for `build_docs.sh --pdf`.

## 9.1 Equation conventions

Dollar math (`myst_enable_extensions = ["dollarmath"]`):

Start a positive closure at its asserted input facts, not at the empty set:

$$I_0 = F, \quad I_{n+1} = I_n \cup \{h\sigma \mid (B \Rightarrow h) \in P, B\sigma \subseteq I_n\}.$$

Write membership explicitly rather than overloading a pair as an RDF triple:

$$\frac{(x, \mathrm{rdf:type}, C)}{(x, \mathrm{rdf:type}, D)} \quad (C \sqsubseteq D).$$

For positive temporal supports, intersect premise validity per derivation, then union the alternative derivations. Entire-window truth is coverage:

$$\mathrm{holdsThroughout}(s, W, h) \iff W \subseteq V_s(h).$$

Here `V_s(h)` is the selected positive truth-coverage set at system horizon `h`, not necessarily one assertion version's interval. See chapter 5 for boundaries.

CHAPTER

TEN

# TUTORIAL NOTEBOOKS

Use the environment described in *Gestalt Graph Onto-Tutorial*. Download or open each lab:

- `nb01_subclass_closure.ipynb` — compile `SubClassOf + Domain`, evaluate with `DatalogEngine`, inspect `result.explain`.
- `nb02_property_chains.ipynb` — binary `PropertyChain` + transitivity, origin table back to axiom IDs.
- `nb03_graph_projection.ipynb` — project a pinned LMDB property graph to facts, infer membership, and visualize a caller-owned logical graph without modifying stored labels.

Notebook outputs are intentionally uncommitted. Tests execute every code cell in order with a fresh namespace and fail on exceptions or failed assertions; they do not merely syntax-check the JSON. The HTML build offers downloads, not a notebook execution service.

CHAPTER

ELEVEN

# OUTLINE COVERAGE

Tracked in GitHub issue #273. Unchecked rows are full-outline acceptance items, not claims that no draft exists. A Sphinx build is not semantic or executable-example verification.

Checklist for the `Gestalt-graph-onto-tutorial` outline. Updated per commit; each row names the doc file, the epic/issue anchor, and the verification.

- [ ] ch01 property-graph vs semantic — #156 context, `graphdb.py`, `ontology/adapters/rdf.py` — `ch01-property-graph-vs-semantic.rst`
- [ ] ch02 fragment + compiler — `ontology/model.py`, `ontology/compiler.py`, gated profiles #184 — `ch02-ontology-fragment.rst`
- [ ] ch03 Datalog IRs — `reasoning/model.py`, `relational.py`, `batches.py`, `plans.py`, `executors.py`, RE-01..RE-06 (#162–#167) — `ch03-datalog-irs.rst`
- [ ] ch04 storage grounding — `extraction.py`, `adapters/batched.py`, `native.py` (#194), `adapters/graph.py` — `ch04-storage-grounding.rst`
- [ ] ch05 claims/temporal/modal — `versioning.py`, `temporal.py`, `epistemic.py`, `modal.py`, RE-E4 (#160) — `ch05-claims-temporal.rst`
- [ ] ch06 query/sampling — Cypher, `ontology-graph.rst`, `SamplerSnapshot/Engine`, #255/#256 — `ch06-query-sampling.rst`
- [ ] ch07 performance — RE-03/04/05/18/22 evidence, cooperative cancel — `ch07-performance.rst`
- [ ] ch08 gaps — #194, #200, #248–#254, RE-E5 (#161), PropEnh #124–#128 — `ch08-gaps.rst`
- [ ] toolkit (figures/equations/snippets/viz) — `viz/api.py` caps, `myst dollarmath`, `mathjax` — `toolkit/`
- [ ] notebooks nb01–nb03 — `pytest tests/test_ontology_tutorial.py`, `sphinx -W` — `notebooks/`

## 11.1 Review corrections (2026-10-22)

- [x] Initial drafts committed and pushed before review.
- [x] Distinguish cross-technology comparisons from supported GestaltDB endpoints.

- [x] Seed closure math with input assertions; account for unions of temporal supports.
- [x] Separate available implementations, experimental contracts, and active acceptance work.
- [x] Document setup, expected outcomes, temporary stores, and notebook downloads.
- [x] Add tests executing chapter code and notebook cells, and reproducing all figures.

## 11.2 Graphics and PDF (2026-10-22)

- [x] Replace tiny circles with large rounded, padded nodes and bold wrapped names.
- [x] Use automatic layered layout, crossing reduction, conceptual clusters, and curved splines.
- [x] Preserve GestaltDB node/edge data and engine-computed memberships; distinguish inferred edges.
- [x] Add a full-resolution overview download and selectable vector text.
- [x] Add `build_docs.sh --pdf` for a tutorial-only XeLaTeX PDF with vector diagrams.
- [x] Document optional build dependencies and test flag handling and missing-tool errors.